KERNEL TRAFFIC ANALYZER

A Production-Grade Linux Network Observability System

A Final Year Project Report

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology in Computer Science & Engineering

Submitted By

Anil Kumar Reddy

Under the Guidance of

Prof. [Supervisor Name]

Department of Computer Science & Engineering

[Your University / Institution Name]

Bhopal, Madhya Pradesh, India

Academic Year 2024–2025

TABLE OF CONTENTS

Chapter 1 Introduction & Objectives

1.1 Background and Motivation

1.2 Problem Statement

1.3 Objectives of the Project

1.4 Scope and Limitations

1.5 Organisation of the Report

1.6 Significance of the Project

1.7 Definitions and Abbreviations

1.8 Chapter Summary

Chapter 2 Literature Survey

2.1 Network Monitoring Approaches

2.2 Kernel-Level Packet Capture Techniques

2.3 DNS Analysis and Threat Detection

2.4 Geographic Routing and BGP Monitoring

2.5 Anomaly Detection in Network Traffic

2.6 Comparative Study of Monitoring Architectures

2.7 Review of Qt-Based Network Tools

2.8 Review of Process Attribution Techniques

2.9 Summary of Related Work

Chapter 3 Research Gap & Proposed System

3.1 Identified Research Gaps

3.2 Proposed System Architecture

3.3 System Design and Modules

3.4 Tools and Technologies Used

3.5 Detailed Data Flow Diagram

3.6 /proc File Interface Design

3.7 Netlink Protocol Design

3.8 Anomaly Detection Algorithm Design

3.9 DNS Pipeline Design

3.10 Security Considerations

Chapter 4 Results and Discussions

4.1 Experimental Setup

4.2 Connection Tracking Results

4.3 Anomaly Detection Performance

4.4 Route Tracing and Geo-Enrichment

4.5 GUI Performance and Usability

4.6 DNS Resolution Performance

4.7 Memory and Resource Usage Analysis

4.8 Comparison with Existing Tools

4.9 Case Study 1: Detecting a Port Scan

4.10 Case Study 2: High Bandwidth Exfiltration

4.11 Case Study 3: BGP Route Change Detection

4.12 Limitations Observed During Testing

4.13 Discussion and Analysis

Chapter 5 Conclusions and Future Work

5.1 Conclusions

5.2 Future Work

5.3 Lessons Learned

5.4 Project Timeline

5.5 Comparison with Project Objectives

5.6 Final Remarks

Chapter 6 Implementation Details

Chapter 7 System Testing and Validation

References

Appendix A Installation Guide

Appendix B /proc File Column Reference

Appendix C Netlink Message Structure Reference

CHAPTER 1 Introduction & Objectives

1.1 Background and Motivation

In the modern computing landscape, network connectivity is ubiquitous and indispensable. Every process on a Linux system — whether a user application, a system daemon, or a background service — routinely establishes network connections to communicate with remote hosts. While this connectivity enables powerful functionality, it also creates significant challenges for system administrators, security analysts, and developers who need to understand precisely what is happening at the network level. The sheer volume of concurrent connections observed on a modern workstation — hundreds of simultaneous TCP sessions across dozens of processes — makes manual inspection infeasible.

The Linux networking stack has evolved considerably since the original socket API was introduced. Modern kernels implement a fully asynchronous, multi-queue networking subsystem that processes packets at line rate using NAPI (New API) polling, hardware offload features such as TSO/GSO/GRO, and receive-side scaling (RSS) across CPU cores. Netfilter, introduced by Rusty Russell in 1998 and shipped as part of the mainline kernel since 2.4, provides five hook points in the IPv4 and IPv6 packet processing paths that allow kernel modules to inspect and modify every packet. These hooks form the foundation of iptables, nftables, and — crucially — KTA.

The case for operating at the kernel level rather than in userspace is compelling from both a completeness and a performance perspective. Userspace tools that read /proc/net/tcp operate on a snapshot that is already stale by the time the read completes; connections that open and close within a single polling interval are permanently lost. In contrast, a netfilter hook sees every packet before any userspace process has a chance to read a socket, providing true completeness. Furthermore, the kernel can aggregate statistics across hundreds of connections with microsecond latency that no userspace polling mechanism can match.

The evolution from BPF (Berkeley Packet Filter, 1993) to eBPF (extended BPF, introduced in kernel 3.18 in 2014) represents the state of the art in kernel-level observability. eBPF allows verified bytecode programs to be injected into the kernel at runtime, attaching to tracepoints, kprobes, and network events without requiring a kernel module. Tools such as Cilium, Falco, and bpftrace are built on eBPF. However, eBPF programs face strict limits: the verifier rejects programs with more than one million instructions (pre-kernel 5.1), loops are constrained, and state must be managed through kernel-side maps. For a complex monitoring suite that maintains connection tables, DNS caches, anomaly

detectors, and route stores, a kernel module written in C offers cleaner state management, unrestricted loop constructs, and direct access to all kernel data structures — at the cost of requiring more rigorous development discipline.

The Kernel Traffic Analyzer (KTA) was conceived and implemented during the final year of a B.Tech Computer Science & Engineering programme at a time when the author was working through a 4G mobile hotspot in Bhopal, Madhya Pradesh, India. The experience of being unable to determine which applications were consuming the limited mobile data allocation, which countries the data was transiting, and whether any anomalous activity was occurring in the background provided the immediate motivation for the project. KTA v7.0 represents approximately twelve thousand lines of code across C, C++, and Python — the culmination of seven major development iterations — and addresses all five research gaps identified in the literature.

1.2 Problem Statement

Despite the maturity of the Linux networking ecosystem, no single open-source tool available as of 2024 simultaneously provides kernel-level connection tracking, per-process attribution, real-time DNS correlation, geographic route visualisation, anomaly detection, and integrated remediation. This gap is not merely inconvenient — it has real security consequences. Industry reports from Verizon (DBIR 2023) indicate that 74% of breaches involved the human element, but a significant fraction of the remainder involved malware or compromised software that exfiltrated data silently over months. In such scenarios, the ability to immediately attribute every outbound byte to a specific process and destination country could dramatically reduce dwell time — the median time between breach and detection, which the 2023 IBM Cost of a Data Breach Report places at 204 days.

Consider a concrete scenario: a developer installs an npm package that includes a supply-chain attack payload. The payload opens a reverse shell to a C2 server in a foreign country. With existing tools, the developer would need to cross-reference netstat output (showing an open TCP connection), DNS resolver logs (showing a suspicious domain resolution), and router logs (showing outbound traffic) to detect the activity. KTA surfaces this in a single screen: the Connections tab shows the node process with ESTABLISHED state, the DNS Map tab shows the domain resolved seconds before the connection, the Route Map animates the hop path through foreign ASNs, and the Anomalies tab may flag HIGH_BW if data is being exfiltrated at speed.

The following six critical deficiencies in existing tools are addressed by KTA:

- No kernel-level visibility: Userspace tools like ss and netstat only read /proc/net/tcp and miss packets that bypass the socket layer or belong to short-lived connections. High-frequency connections that open and close within one polling cycle are silently dropped from the monitoring record entirely.
- Lack of per-process attribution: Raw packet capture tools such as tcpdump operate at the interface level and do not attribute traffic to specific OS processes or executable paths.
- No real-time DNS correlation: Knowing that a connection goes to 142.250.77.46 is far less meaningful than knowing it goes to google.com. DNS correlation must happen at packet interception time to be useful.
- No geographic routing awareness: Administrators cannot easily determine which countries their data transits through, nor detect unexpected routing changes that could indicate BGP hijacking or route leaks.
- Fragmented anomaly detection: Security teams rely on separate SIEM tools for anomaly detection, creating latency between an event and its detection. Kernel-level detection with sub-second latency is achievable.
- No integrated firewall control: Blocking a suspicious connection typically requires switching to a terminal and writing iptables rules manually, interrupting the monitoring workflow.

1.3 Objectives of the Project

Seven primary objectives were defined at the outset of the project. Each objective corresponds to a specific technical gap identified in Section 1.2 and is directly traceable to an implemented subsystem in KTA.

Objective 1 — Kernel-level packet interception: Design and implement a Linux kernel module that intercepts all TCP and UDP traffic at the NF_INET_LOCAL_IN and NF_INET_LOCAL_OUT netfilter hooks without dropping any packets. The hooks must operate at NF_IP_PRI_FIRST priority to ensure they run before connection tracking, and must return NF_ACCEPT unconditionally so that network functionality is not impaired.

Objective 2 — Per-process attribution: Implement per-process attribution by correlating kernel sockets to PIDs using a multi-level cache hierarchy (sock_cache, inode_cache, flow_cache, full resolver). The target attribution accuracy is above 95% for TCP connections with duration exceeding 500 ms.

Objective 3 — In-kernel DNS parsing: Parse DNS responses in RFC 1035 wire format within the netfilter hook to build a real-time IP→domain mapping cache available at connection creation time.

Objective 4 — Route daemon with geo-enrichment: Integrate a Python-based route daemon that performs TCP traceroute on port 443 and enriches each hop with MaxMind GeoLite2 city, country, latitude/longitude, ASN, and organisation data.

Objective 5 — Anomaly detection: Implement five lightweight anomaly detection algorithms (port scan, SYN flood, connection burst, high connection count, high bandwidth) operating entirely within the kernel using per-process velocity trackers.

Objective 6 — Proc interface and Qt6 GUI: Expose all collected data through seven /proc file interfaces consumed by a Qt6 GUI refreshing every second across 15 specialised monitoring tabs.

Objective 7 — Integrated remediation: Provide one-click firewall control (iptables), bandwidth throttling (tc-netem), data export to JSON/CSV/PDF, and a system tray with anomaly badge notifications.

1.4 Scope and Limitations

The scope of this project covers IPv4 and partial IPv6 (last 32 bits stored) TCP and UDP traffic on Ubuntu 22.04 LTS with Linux kernel 6.8.0. The system monitors the local machine only and does not perform deep packet inspection beyond DNS response parsing.

The following limitations were accepted as part of the project scope:

- Full IPv6 128-bit address tracking is not implemented. The flow_key struct stores only 32 bits of the IPv6 source and destination address. This is flagged as future work.
- Route tracing relies on traceroute which requires root privileges. On networks where TCP SYN on port 443 is rate-limited, hop discovery may be incomplete.
- The connection table is capped at 2,048 entries. Under sustained high-traffic conditions (observed peak: 1,847 entries), older CLOSED entries are evicted.
- MaxMind GeoLite2 databases require a free account registration and periodic manual updates. City-level accuracy varies by region; ASN-level accuracy is typically above 99%.
- The SparklineDelegate in the Qt6 GUI must not inherit QObject due to a vtable linker conflict with QStyledItemDelegate. This is a known Qt limitation.

1.5 Organisation of the Report

The remainder of this report is structured as follows. Chapter 2 presents a comprehensive literature survey. Chapter 3 identifies the research gaps and presents the proposed system architecture. Chapter 4 presents experimental results. Chapter 5 draws conclusions and outlines future directions. Chapter 6 provides implementation details for key subsystems. Chapter 7 describes the testing and validation approach.

1.6 Significance of the Project

KTA represents a contribution to the open-source network security and observability ecosystem that is distinctive in its combination of depth and breadth. While individual capabilities such as kernel packet capture or DNS monitoring exist in separate tools, no comparable open-source project integrates all of them in a single installable kernel module with a production-quality GUI.

From an educational perspective, KTA serves as a reference implementation for several advanced Linux kernel programming techniques: softirq-safe spinlock usage (spin_lock_bh), RCU read-lock protected task structure access, GFP_ATOMIC memory allocation in interrupt context, workqueue-based deferred work scheduling, and /proc filesystem virtual file implementation.

From a practical security perspective, KTA addresses a real need in the Indian developer and SME community where dedicated network monitoring appliances are cost-prohibitive. A free, open-source tool that runs on commodity hardware with a 4G internet connection provides visibility previously available only to enterprises with dedicated SOC (Security Operations Centre) infrastructure.

1.7 Definitions and Abbreviations

Term / Abbreviation	Definition
ASN	Autonomous System Number — a globally unique identifier assigned to a collection of IP address ranges under unified administrative control.
BPF / eBPF	Berkeley Packet Filter / extended BPF — a kernel virtual machine that executes sandboxed programs in kernel context for packet filtering and observability.
BGP	Border Gateway Protocol — the inter-domain routing protocol of the internet, used by ASNs to advertise IP prefix reachability.
CLOSED	TCP connection state after both FIN/ACK exchange is complete and the connection entry is eligible for eviction.
DNS	Domain Name System — the hierarchical naming system that maps human-readable domain names to IP addresses.
FIN_WAIT	TCP state after the local endpoint has sent FIN, awaiting the remote FIN+ACK.
flow_key	KTA internal struct: {src_ip, dst_ip, src_port, dst_port, proto} identifying a unique 4-tuple

Term / Abbreviation	Definition
	after make_canonical() normalisation.
GFP_ATOMIC	Linux kernel memory allocation flag meaning 'do not sleep' — required for all allocations in softirq/interrupt context.
GeoLite2	MaxMind's free-tier IP geolocation database providing city, country, and ASN data.
GUI	Graphical User Interface — in KTA context, the Qt6 C++ application.
inode_cache	KTA second-level cache mapping socket inode numbers to PIDs.
iptables	Linux userspace firewall administration tool that manages netfilter rules.
KTA	Kernel Traffic Analyzer — the system described in this report.
NF_ACCEPT	Netfilter verdict meaning 'continue normal packet processing' — returned by all KTA hooks.
NF_INET_LOCAL_IN	Netfilter hook point for packets destined to the local machine (post-routing).
NF_INET_LOCAL_OUT	Netfilter hook point for packets generated by local processes (pre-routing).
NF_IP_PRI_FIRST	Highest priority value (-300) for netfilter hook registration — ensures KTA runs before conntrack.
PID	Process Identifier — a kernel-assigned integer uniquely identifying a running process.
proc_fs	Linux pseudo-filesystem mounted at /proc, exposing kernel data structures as virtual files.
RFC 1035	IETF standard defining the DNS wire format for queries and responses.
RCU	Read-Copy-Update — a synchronisation mechanism for read-mostly data structures in the kernel.
skb	Socket buffer (struct sk_buff) — the primary kernel data structure for network packets.
sock_cache	KTA first-level cache mapping struct sock pointers directly to PIDs.
SYN_SENT	TCP state after the client has sent SYN, before receiving SYN+ACK.

Term / Abbreviation	Definition
tc / tc-netem	Linux Traffic Control — kernel subsystem for packet scheduling, shaping, and emulation.
TTL	Time To Live — a field in IP headers limiting hop count; also used in DNS records to specify cache lifetime.
UDP	User Datagram Protocol — connectionless transport protocol used for DNS, streaming, and other latency-sensitive applications.
workqueue	Linux kernel deferred work mechanism that executes functions in a kernel thread context.

Table 1.1 — Definitions and Abbreviations

1.8 Chapter Summary

This chapter established the motivation for KTA by highlighting the limitations of existing network monitoring tools and the real security implications of inadequate network visibility. The problem statement was articulated as six specific deficiencies in existing tools. Seven project objectives were defined, each with a clear design rationale traceable to an implemented subsystem. The scope and limitations were defined honestly, including incomplete IPv6 support and the 2,048-connection table cap. Chapter 2 now surveys the existing literature to establish the academic context for these contributions.

CHAPTER 2 Literature Survey

This chapter surveys existing work in the areas of network traffic monitoring, kernel-level packet capture, DNS-based analysis, geographic routing, anomaly detection, and related GUI frameworks. The survey establishes the academic and technical context for the research gaps addressed by KTA and justifies the design decisions made in Chapter 3.

2.1 Network Monitoring Approaches

2.1.1 Traditional Userspace Tools

tcpdump, originally developed by Van Jacobson, Craig Leres, and Steven McCanne at Lawrence Berkeley National Laboratory in 1989, pioneered kernel-assisted packet capture through the Berkeley Packet Filter mechanism. BPF allows filter expressions to be compiled into a bytecode program that runs in kernel space to decide which packets to copy to userspace, dramatically reducing overhead compared to copying all packets and filtering in userspace. tcpdump remains the de facto standard for command-line packet capture. However, it does not associate packets with processes and requires substantial manual effort to correlate output with application behaviour.

Wireshark (originally Ethereal, renamed in 2006) was created by Gerald Combs in 1998 and has since grown into the most widely used graphical packet analyser in the world. Wireshark dissects over 3,000 protocols, supports live capture via pcap and libpcap, and provides a powerful display filter language. However, it is not designed for continuous production monitoring: live capture at high packet rates consumes substantial CPU and memory, and the interface becomes unwieldy when thousands of connections are open simultaneously.

ss (socket statistics) and its predecessor netstat are included in every major Linux distribution. ss reads from kernel socket diag infrastructure (SOCK_DIAG, RFC 7395) which provides more detailed information than /proc/net/tcp but still operates on a point-in-time snapshot. The fundamental limitation is that ss shows only currently open connections; any connection that opened and closed between two ss invocations is invisible.

nethogs (Arnout Engelen, 2004) provides per-process bandwidth monitoring by parsing /proc/net/tcp periodically and correlating with socket inodes from /proc/[pid]/fd. It suffers from the same snapshot limitation as netstat, cannot show connection destinations or DNS names, provides no anomaly detection, and has no history or export capability.

2.1.2 Flow-Based and eBPF Monitoring

NetFlow, developed by Cisco in 1996, exports flow summaries from routers and switches to collection servers. NetFlow provides excellent scalability for network-wide visibility but operates at the network device level and does not attribute flows to OS processes.

Extended Berkeley Packet Filter (eBPF), mainlined in kernel 3.18 (2014), has emerged as the dominant paradigm for kernel-level observability without writing a traditional kernel module. Tools built on eBPF include Cilium, Falco, Pixie, and bpftrace. Despite its power, eBPF has limitations relevant to KTA's design — complexity limits constrain per-packet processing, and CO-RE portability adds toolchain complexity.

2.2 Kernel-Level Packet Capture Techniques

The Linux Netfilter framework, introduced by Rusty Russell in 1998, provides five hook points in the IPv4 packet processing path. KTA registers at LOCAL_IN and LOCAL_OUT to capture all traffic to and from the local machine. Hook registration uses nf_hook_ops structs specifying the hook number, priority, protocol family (NFPROTO_IPV4), and callback function pointer. KTA registers at NF_IP_PRI_FIRST (-300), which is lower than the connection tracking priority (NF_IP_PRI_CONNTRACK = -200), ensuring KTA sees the original pre-NAT packet.

PID attribution from a netfilter hook presents a significant challenge: the hook runs in softirq context, where current may or may not point to the process that generated the packet. KTA handles this asymmetry by using direct current->pid for outgoing packets when the socket is accessible, and relying on the cache hierarchy for all other cases, with async workqueue resolution for cases that miss all caches.

A critical implementation detail is the use of rcu_dereference(mm->exe_file) to obtain the executable file path for a process. In Linux kernel versions prior to 6.7, this was done via get_mm_exe_file(). That function was removed in kernel 6.7 (commit 47b9f88); KTA uses the RCU-protected direct access pattern, holding the RCU read lock with rcu_read_lock() around the dereference.

2.3 DNS Analysis and Threat Detection

DNS is the foundational naming layer of the Internet. Paul Mockapetris designed DNS in RFC 882 (1983) and the final wire format specification in RFC 1035 (1987). Understanding this format —

including pointer compression, label encoding, and the structure of resource records — is essential for implementing a correct in-kernel parser.

Passive DNS collection, pioneered by Florian Weimer at CERT/Bund and presented at the FIRST conference in 2005, records all DNS responses observed on a network segment and builds a historical database of IP-to-domain mappings. KTA's dns_map implements a lightweight single-host version of this concept.

DNS-based exfiltration is a well-documented attack technique in which an attacker encodes data in DNS query labels to bypass firewalls that allow DNS traffic but block direct TCP connections. While KTA's current implementation does not implement DNS exfiltration detection, the dns_map and DNS leak detection subsystems provide the foundation for adding it.

2.4 Geographic Routing and BGP Monitoring

MaxMind was founded in 2002 and has since become the dominant provider of IP geolocation data. The GeoLite2 databases (City and ASN variants) are freely available under a Creative Commons Attribution-ShareAlike 4.0 licence. The GeoLite2-City database maps IPv4 and IPv6 addresses to city, country, ISO country code, latitude, and longitude with an accuracy of approximately 55–80% at the city level depending on region. ASN accuracy is above 99% for publicly routable addresses.

The traceroute utility, written by Van Jacobson in 1987, discovers the hop-by-hop path between two hosts by sending probes with increasing TTL values. TCP traceroute (traceroute -T) sends TCP SYN packets on a specified port. Port 443 (HTTPS) is chosen by KTA because it is the least likely to be blocked on any modern network, particularly through carrier-grade NAT in 4G environments.

BGP hijacking represents one of the most serious threats to internet routing integrity. Notable historical incidents include the Pakistan Telecom hijacking of YouTube's prefix in 2008 and the Rostelecom BGP incident in 2020. KTA's BgpMonitor implements a lightweight endpoint-perspective BGP change detector using a sliding 7-day baseline window.

2.5 Anomaly Detection in Network Traffic

Network anomaly detection has been studied since Denning's seminal 1987 paper on intrusion detection systems. Threshold-based detection remains widely used in production systems because it is computationally efficient, deterministic, and easy to tune. KTA implements five such detectors entirely within the kernel, operating on per-process velocity counters updated on every packet.

Port scan detection: The classic threshold approach described by Staniford, Hoagland, and McAlerney (2002) uses a sliding window over the number of unique destination ports contacted by a single source. A threshold of 15 unique destination ports per second is consistent with conservative enterprise policies — nmap's default SYN scan rate contacts approximately 1,000 ports per second, making this threshold trivially detectable within the first second of a scan.

Machine learning approaches to network anomaly detection include Isolation Forest (Liu et al., 2008) and LSTM-based models (Mirsky et al., 2018 — Kitsune). These approaches require training data and periodic model updates but achieve lower false positive rates for novel attack patterns. KTA's Section 5.2 identifies ML-based anomaly detection as a priority future work item.

2.6 Comparative Study of Monitoring Architectures

Architecture	Examples	Capture Point	State	Presentation	Process Attr.	DNS Corr.	Geo Route
Packet sniffer	tcpdump, tshark	Kernel (BPF)	Stateless	CLI	No	No	No
GUI sniffer	Wireshark	Kernel (BPF)	Stateless	GUI	Partial	Yes	No
Proc poller	netstat, ss, nethogs	Userspace	Snapshot	CLI	Limited	No	No
Flow exporter	NetFlow, sFlow	Network device	Stateful	Collector	No	No	No
eBPF tracer	Cilium, Falco	Kernel (eBPF)	Stateful (maps)	API/CLI	Yes	Yes	No
KTA (this work)	**Kernel Module**	**Kernel (Netfilter)**	**Stateful (hash)**	**GUI + /proc**	**Yes (4-level)**	**Yes**	**Yes**

Table 2.1 — Comparative study of monitoring architectures

2.7 Review of Qt-Based Network Tools

The Qt framework has a long history of use in network monitoring tools. Qt's signal-slot mechanism, model-view-delegate architecture, and cross-platform widget set make it well-suited for data-rich GUIs that update in real time. Wireshark's GUI is built on Qt (migrated from GTK+ in version 1.10, 2013). KTA's TrafficModel and ProcModel follow this pattern directly, using

QAbstractTableModel for efficient data updates and QStyledItemDelegate for custom sparkline cell rendering.

2.8 Review of Process Attribution Techniques in Linux

The Linux kernel's SOCK_DIAG interface provides a netlink-based API for querying socket information including inode numbers. Userspace tools use this to build an {inode → PID} mapping by cross-referencing with /proc/[pid]/fd entries. This approach is used by ss, nethogs, and Wireshark's extcap interface. The limitation is that it is a polling-based approach with inherent race conditions.

eBPF-based attribution using kprobes on tcp_v4_connect, inet_csk_accept, and udp_sendmsg system calls provides exact attribution by capturing the current task_struct at the moment of socket creation or first use. KTA's four-level cache hierarchy represents a pragmatic compromise: the sock_cache provides O(1) lookup for sockets that have already been resolved (71.2% hit rate in testing), and the full resolver is triggered only when all caches miss (1.1% of connections).

2.9 Summary of Related Work

Tool	Kernel Cap.	Per-Process	DNS Corr.	Geo Route	Anomaly	Firewall	GUI	
tcpdump	Yes	No	No	No	No	No	No	
Wireshark	Yes	Partial	Yes	No	No	No	Yes	
netstat/ss	No	Yes	No	No	No	No	No	
nethogs	No	Yes	No	No	No	No	No	
iftop	Yes (BPF)	No	Yes	No	No	No	Ncurses	
Cilium/eBPF	Yes	Yes	Yes	No	Yes	Yes	API only	
Falco	Yes	Yes	No	No	Yes	No	No	
GlassWire	Yes (Win)	Yes	Yes	No	Limited	Limited	Yes	
KTA (Ours)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	

Table 2.2 — Comprehensive feature comparison of network monitoring tools

CHAPTER 3 Research Gap & Proposed System

3.1 Identified Research Gaps

Based on the literature survey presented in Chapter 2, five research gaps are identified that motivate the design of KTA:

- Gap 1 — Unified real-time enrichment: No existing open-source tool provides simultaneous kernel-level capture, per-process attribution, DNS correlation, and geographic enrichment in a single pipeline with sub-second latency.
- Gap 2 — In-kernel DNS parsing: Existing tools either capture DNS at the resolver level (missing rogue direct DNS queries) or post-process PCAP files. Parsing DNS responses in the netfilter hook before connection tracking runs ensures immediate domain availability at connection creation time.
- Gap 3 — TCP traceroute integration: No monitoring GUI performs live TCP-mode traceroute to each new destination and renders the hop path geographically. Carrier-grade NAT in 4G environments blocks ICMP, making TCP mode on port 443 essential.
- Gap 4 — Push-based netlink event stream: Poll-based /proc reading creates up to 1-second latency and unnecessary CPU overhead. A push-based netlink multicast channel enables real-time GUI updates on connection state changes and anomaly events.
- Gap 5 — Integrated remediation: No existing monitoring tool integrates firewall rule management and bandwidth throttling within the monitoring interface itself.

3.2 Proposed System Architecture

KTA is structured as three cooperating layers. Each layer has clearly defined responsibilities and communicates with adjacent layers through well-defined interfaces.

Layer	Technology	LOC	Interface	Responsibility
Layer 1: Kernel Module	C (GCC)	~3,500	7 /proc files + netlink multicast	Packet capture, PID attribution, DNS parsing, anomaly detection
Layer 2: Route Daemon	Python 3	~330	/proc/traffic_analyzer_routes	TCP traceroute, MaxMind GeoLite2 enrichment, hop store population
Layer 3: Qt6 GUI	C++ (Qt6)	~8,000	Reads /proc files, subscribes to netlink	15 monitoring tabs, SQLite history, iptables/tc control, PDF/JSON

Layer	Technology	LOC	Interface	Responsibility
				export

Table 3.1 — KTA three-layer architecture summary

3.3 System Design and Modules

3.3.1 Kernel Module Subsystems

The kernel module is decomposed into 12 source files, each with a single well-defined responsibility.

Source File	Lines	Responsibility
module_main.c	120	Module entry/exit, initialises subsystems in dependency order
netfilter_hook.c	180	Two nf_hook_ops, DNS interception before parse_packet()
packet_parser.c	450	10-step packet processing pipeline, PID attribution waterfall
stats.c	520	TCP FSM, rate window, velocity tracker, anomaly detection, netlink throttle
proc_interface.c	380	7 /proc file creation, unified write handler, control commands
dns_parser.c	200	RFC 1035 wire format parser, pointer compression, A record extraction
dns_map.c	150	IP→domain hash cache, TTL tracking, LRU eviction
route_store.c	280	jhash table for hop data, pending_write_ip state machine
resolver.c	220	ta_resolver workqueue (4 threads), socket and exe path resolver
sock_cache.c	160	struct sock* → PID cache, RCU-safe operations
inode_cache.c	140	Socket inode → PID cache
flow_cache.c	160	5-tuple flow_key → PID cache
netlink_comm.c	190	NETLINK_USERSOCK proto 2, 4 multicast groups, 6 message types

Table 3.2 — Kernel module source files and responsibilities

3.3.2 Route Daemon Design

The Python route daemon (ta_route_daemon.py, approximately 330 lines) bridges the kernel module and the MaxMind databases. It operates as a long-running system service with no external dependencies beyond the geoip2 Python package and the standard traceroute utility. The daemon opens the GeoLite2-City and GeoLite2-ASN database files once at startup using the geoip2.database.Reader context manager, avoiding the overhead of repeated file opens. The main loop polls /proc/traffic_analyzer_routes_pending every 2 seconds.

3.3.3 GUI Architecture

The Qt6 GUI (approximately 8,000 lines of C++) is structured around a QMainWindow with a QStackedWidget for the 15 content tabs and a custom CollapsibleSidebar widget. A single QTimer with a 1,000 ms interval drives all data refreshes. The refresh cycle calls ProcReader::readAll() which opens and reads all five primary /proc files sequentially; the total read time observed in testing was consistently under 5 ms even with the maximum 2,048 connection entries.

3.4 Tools and Technologies Used

Component	Technology	Version	Purpose
Kernel Module	C (GNU C)	GCC 11.4	Netfilter hooks, proc interface, DNS parser
Build System (Kernel)	Makefile + Kbuild	Kernel 6.8	Kernel module compilation
GUI	Qt6 / C++	Qt 6.4.2	Cross-platform graphical interface
GUI Build	CMake	3.22+	GUI compilation with Qt6 find_package
Route Daemon	Python 3	3.10.12	Traceroute orchestration and GeoIP enrichment
GeoIP	MaxMind GeoLite2	Jan 2025	City, country, ASN, lat/lon lookup
History DB	SQLite3	3.37	Per-process bandwidth history
Packet Trace	traceroute	2.1.0	TCP traceroute -T -p 443
OS	Ubuntu 22.04 LTS	22.04.3	Target deployment platform

Component	Technology	Version	Purpose
Kernel Target	Linux 6.8.0-106	—	Netfilter, RCU, workqueue APIs

Table 3.3 — Technology stack

3.5 Detailed Data Flow Diagram

#	Component	Input	Output	Latency	Notes
1	Packet arrival / Kernel netfilter	Raw skb from NIC	skb at hook	<1 µs	Hardware DMA → ring buffer
2	DNS check / netfilter_hook.c	UDP/53 skb	IP→domain in dns_map	<2 µs	Phase 1 DNS pipeline
3	Packet parse / packet_parser.c	skb at hook	flow_key, protocol info	1–3 µs	10-step pipeline
4	PID resolution / resolver.c (async)	flow_key	PID + exe path	0–100 ms	4-level cache waterfall
5	Stats update / stats.c	flow_key + PID	connection entry, anomaly flags	<1 µs	Velocity trackers updated
6	Proc export / proc_interface.c	connection table	/proc files on read	on demand	seq_file virtual files
7	Route request / proc_interface.c	new dest IP	PENDING in route_store	<1 µs	Triggers route daemon
8	Route daemon / ta_route_daemon.py	PENDING IP	traceroute + GeoIP data	10–60 s	TCP mode port 443
9	Route write / proc_interface.c	write DEST/HOP lines	hop data in route_store	<1 µs	Protocol parsed in kernel
1	GUI refresh /	/proc files	data model update	1 s interval	readAll() < 5 ms

#	Component	Input	Output	Latency	Notes
0	ProcReader.cpp				
1 1	GUI render / TrafficModel	data models	screen pixels	<5 ms	Qt paint event
1 2	Netlink event / netlink_comm.c	connection change	multicast message	<1 µs	Throttled per connection
1 3	GUI netlink / NetlinkListener	netlink message	immediate GUI update	<10 ms	Push notification

Table 3.4 — KTA data flow pipeline stages

3.6 /proc File Interface Design

The seven /proc files exposed by KTA form the primary interface between the kernel module and the GUI. Each file is implemented as a seq_file backed virtual file in proc_interface.c. All files use pipe-delimited (|) columns for easy parsing. The following sections describe the main connection table file in detail.

3.6.1 /proc/traffic_analyzer — Main Connection Table

This is the primary output file, containing one row per tracked TCP/UDP connection. It is read by ProcReader::readConnections() every 1 second. The 22 columns are defined as follows:

Co l	Name	Type	Description
1	PID	Integer	Process ID of the attributed process. 0 if unresolved.
2	UID	Integer	User ID of the process owning the socket.
3	PROCESS	String	Process name (comm field, up to 15 chars).
4	RESOLVED	Boolean (0/1)	1 if PID attribution is confirmed, 0 if pending.
5	STATE	String	TCP state: SYN_SENT, ESTABLISHED, FIN_WAIT, CLOSED, or

Col	Name	Type	Description
			UDP.
6	DNS	Boolean (0/1)	1 if the destination domain was resolved from DNS cache.
7	PROTO	String	Protocol: TCP or UDP.
8	SRC_IP	IPv4 dotted	Source IP address.
9	DEST_IP	IPv4 dotted	Destination IP address.
10	SRC_PORT	Integer	Source port number.
11	DEST_PORT	Integer	Destination port number.
12	DOMAIN	String	Domain name from DNS cache, or empty string.
13	OUT_BYTES	Long	Total outgoing bytes since connection creation.
14	IN_BYTES	Long	Total incoming bytes since connection creation.
15	OUT_PKTS	Long	Total outgoing packets.
16	IN_PKTS	Long	Total incoming packets.
17	RATE_OUT_BPS	Long	Current 1-second outgoing rate in bytes/sec.
18	RATE_IN_BPS	Long	Current 1-second incoming rate in bytes/sec.
19	FIRST_SEEN	Long (Unix)	ktime_get_real_seconds() at connection creation.
20	LAST_SEEN	Long (Unix)	ktime_get_real_seconds() at last packet.
21	DURATION	Long (secs)	LAST_SEEN − FIRST_SEEN.
22	CLOSED_AT	Long (Unix)	ktime_get_real_seconds() when CLOSED state was entered, or 0.

Table 3.5 — /proc/traffic_analyzer column definitions

3.7 Netlink Protocol Design

KTA introduces a push-based netlink multicast channel that eliminates the need for the GUI to poll /proc files for every state change. The implementation uses NETLINK_USERSOCK with protocol number 2. Four multicast groups are defined: group 1 (connections), group 2 (anomalies), group 3 (DNS), and group 4 (routes).

Message Type	Group	Struct	Key Fields
CONN_NEW	1 (connections)	ta_msg_conn	pid, uid, process, src_ip, dst_ip, ports, proto, domain
CONN_UPDATE	1 (connections)	ta_msg_conn	Same as CONN_NEW plus state, rate_out_bps, rate_in_bps
CONN_CLOSED	1 (connections)	ta_msg_conn	Same as CONN_NEW plus closed_at timestamp
ANOMALY	2 (anomalies)	ta_msg_anomaly	pid, process, anomaly_type, value, threshold, timestamp
DNS_RESOLVED	3 (DNS)	ta_msg_dns	ip, domain, ttl, queried_by_pid, queried_by_comm
ROUTE_READY	4 (routes)	ta_msg_route	dest_ip, total_hops, hops[32] (inline)

Table 3.7 — Netlink message types and their fields

3.8 Anomaly Detection Algorithm Design

All five anomaly detection algorithms operate on per-process velocity counters maintained in the stats.c velocity_tracker struct. The tracker is updated on every call to stats_update() and maintains a 1-second sliding window using ktime_get_ns() for timestamps.

Algorithm 1: PORT_SCAN Detection

Each process maintains a unique_dstports bitmap (512 bits). On each outgoing TCP SYN packet, the bit corresponding to dst_port % 512 is set. The port count is computed via popcount(). If

port_count >= 15 (ANOMALY_PORT_SCAN_PORTS), the ANOMALY_PORT_SCAN flag is set. The bitmap approach allows constant-time unique port counting and uses only 64 bytes per process.

Algorithm 2: SYN_FLOOD Detection

Each process maintains syn_pending_count and total_tcp_count. A process is flagged when the ratio syn_pending_count / total_tcp_count exceeds 0.80 (ANOMALY_SYN_RATIO) AND total_tcp_count >= 10. The minimum connection count guard prevents false positives immediately after a process starts making its first few connections.

Algorithms 3–5: CONN_BURST, HIGH_CONNS, HIGH_BW

CONN_BURST flags processes that open more than 20 new connections per second. HIGH_CONNS flags processes maintaining more than 200 simultaneous connections. HIGH_BW flags processes with combined in/out rate exceeding 10 MB/s (10,485,760 bytes/sec). These thresholds allow normal browser behaviour (6–15 parallel connections per page load) while flagging clearly automated activity.

3.9 DNS Pipeline Design

KTA's DNS pipeline operates in two phases. Phase 1 runs in the NF_INET_LOCAL_IN hook when a UDP packet arrives from source port 53 (a DNS response). The dns_parser.c receives the UDP payload and parses it according to RFC 1035 wire format, following label pointers with a 10-jump loop guard to prevent pointer-loop denial-of-service attacks. Up to 64 answer records are processed per response.

Phase 2 runs in parse_packet() when a new connection entry is created. When a new traffic_entry is created for a connection, it calls dns_map_lookup(dst_ip) to check if the destination IP was recently resolved. Because DNS responses always precede the TCP SYN that initiates the connection, this lookup succeeds for the vast majority of connections initiated by well-behaved applications — achieving 94.0% immediate domain enrichment.

3.10 Security Considerations

As a kernel module that processes every network packet on the system, KTA requires root privileges for loading. Several security considerations informed the implementation. All netfilter hooks return NF_ACCEPT unconditionally — KTA never modifies, delays, or drops packets. All data

structures accessed from both softirq context and process context are protected with spin_lock_bh(). All dynamic allocations use GFP_ATOMIC in softirq context, with allocation failures handled gracefully. All data structures (connection table, DNS map, route store) are bounded to prevent unbounded memory growth under adversarial traffic conditions.

CHAPTER 4 Results and Discussions

4.1 Experimental Setup

All tests were conducted on a machine connected through a 4G mobile hotspot to simulate the most restrictive network environment targeted by the project. A 72-hour continuous monitoring run was performed.

Parameter	Value
CPU	Intel Core i5-8250U, 4 cores / 8 threads @ 3.4 GHz boost
RAM	8 GB DDR4-2400, dual channel
Storage	256 GB NVMe SSD
OS	Ubuntu 22.04.3 LTS (Jammy Jellyfish)
Kernel	6.8.0-106-generic (HWE stack)
Network	4G LTE hotspot (~30 Mbps down, ~10 Mbps up)
Python	3.10.12
Qt	6.4.2
GeoLite2 DB date	January 2025
traceroute	2.1.0
Test duration	72 hours continuous monitoring
Traffic load (synthetic)	100 Mbps peak (iperf3 loopback for overhead tests)

Table 4.1 — Experimental setup

4.2 Connection Tracking Results

Metric	Value
Total connections tracked	84,312
Unique processes observed	47
PID attribution accuracy (all)	96.8%
PID attribution accuracy (TCP ≥ 500 ms)	99.1%
Connections via sock_cache hit	71.2%
Connections via inode_cache hit	18.4%
Connections via flow_cache hit	6.1%
Connections via full resolver	1.1%
Unresolved (kernel threads + short UDP)	3.2%
Average stats_update() latency (ktime)	< 2 µs
Max traffic_list size (peak load)	1,847 entries
Connection table evictions (72 h)	0 (peak never exceeded 2,048)

Table 4.2 — Connection tracking statistics (72-hour run)

4.3 Anomaly Detection Performance

Anomaly Type	Threshold	Trigger Tool	Detection Latency	False Positive Rate
PORT_SCAN	15 unique dst ports/sec	nmap -sS	1.0 – 1.5 s	0.8%
SYN_FLOOD	SYN ratio ≥ 80%	hping3 --flood	1.0 – 1.2 s	0.3%
CONN_BURST	20 new conns/sec	Custom Python script	1.0 – 1.1 s	1.2%

Anomaly Type	Threshold	Trigger Tool	Detection Latency	False Positive Rate
HIGH_CONNS	200 total connections	Apache Bench (ab)	< 1.0 s	0.1%
HIGH_BW	10 MB/s combined	iperf3	1.0 – 2.0 s	0.5%

Table 4.3 — Anomaly detection evaluation results

4.4 Route Tracing and Geo-Enrichment

Metric	Value
Total destination IPs traced	200
Successful (≥ 1 hop returned)	192 (96%)
Failed (symmetric NAT, no hops)	8 (4%)
Average hop count	12.4 hops
Average traceroute time per destination	28 seconds
GeoIP city resolution rate (public IPs)	87%
ASN resolution rate (public IPs)	98%
Simulated BGP changes detected	2 of 2 (100%)

Table 4.4 — Route tracing and geo-enrichment results

4.5 GUI Performance and Usability

Metric	Value
GUI CPU usage (average)	2.1% on one core
GUI CPU usage (range)	1.2% – 3.5%
GUI memory usage (RSS)	48 MB stable over 8 hours

Metric	Value
SQLite history DB growth	~12 KB / hour
Kernel module CPU overhead	< 0.05% at 100 Mbps
SparklineDelegate render time	< 0.3% CPU
Route map repaint interval	30 ms (smooth animation)
iptables rule apply latency	< 50 ms
tc throttle rule apply latency	< 80 ms
JSON export (150 conns, 200 routes)	< 200 ms
PDF export (full report)	< 1.5 seconds
All 15 tabs functional	Yes

Table 4.5 — GUI performance metrics

4.6 DNS Resolution Performance

Metric	Value
Total unique connections (TCP + UDP)	84,312
Connections with domain at creation time	79,253 (94.0%)
Connections enriched retroactively	1,847 (2.2%)
Connections with no domain observed	3,212 (3.8%)
DNS responses parsed (72 h)	312,847
DNS A records extracted	1,124,392
Average TTL in dns_map at connection time	187 seconds
DNS map peak size	3,847 entries

Metric	Value
DNS pointer decompression errors	0
DNS parse failures (malformed responses)	3 (< 0.001%)

Table 4.6 — DNS resolution performance statistics

4.7 Memory and Resource Usage Analysis

Data Structure	Max Entries	Bytes per Entry	Max Total
traffic_entry (connection table)	2,048	~320 bytes	655 KB
proc_entry (process table)	512	~280 bytes	140 KB
dns_map entry	4,096	~128 bytes	512 KB
route_store hop data	2,048 (32 hops each)	~64 bytes/hop	4 MB
sock_cache	4,096	~32 bytes	128 KB
inode_cache	2,048	~32 bytes	64 KB
flow_cache	2,048	~48 bytes	96 KB
Total (approximate maximum)	**—**	**—**	**~5.6 MB**

Table 4.7 — Kernel module memory footprint at maximum capacity

4.8 Comparison with Existing Tools

Metric	tcpdump	nethogs	ss -tulnp	KTA
CPU overhead	3–8%	0.5–1%	< 0.01% (snapshot)	**< 0.05%**
Memory (process)	~80 MB (PCAP	~8 MB	Negligible	**Kernel: 2.1 MB / GUI: 48**

Metric	tcpdump	nethogs	ss -tulnp	KTA
	buffer)			**MB**
Attribution accuracy	None	~85%	~90%	**96.8%**
Short-lived conn coverage	100% (PCAP)	~60%	~30%	**~97%**
DNS enrichment	Manual post-process	None	None	**94% automatic**
Geo routing	None	None	None	**96% of IPs**
Anomaly detection	Manual	None	None	**5 types, < 2 s**
Firewall integration	None	None	None	**One-click iptables**
Real-time GUI	No	Ncurses	No	**Qt6, 15 tabs**

Table 4.8 — Quantitative comparison with existing tools

4.9 Case Study 1: Detecting a Port Scan

This case study demonstrates KTA's PORT_SCAN anomaly detection in a realistic scenario. An attacker runs nmap -sS 192.168.1.0/24 -p 1-1000 from the monitored machine, generating approximately 254,000 SYN packets. At nmap's default timing, unique destination ports exceed the 15/second threshold within the first second of scanning.

Time (s)	Event	KTA Action
T+0.000	nmap starts, first SYN sent to 192.168.1.1:1	sock_cache miss → workqueue resolver queued
T+0.020	Resolver completes: PID=12847, comm=nmap	sock_cache populated, proc_entry created for nmap
T+0.100	50+ SYN packets sent to unique ports	port_count in velocity_tracker = 50
T+1.000	End of first 1-second window	port_count = 500 >> threshold (15), ANOMALY_PORT_SCAN set

Time (s)	Event	KTA Action
T+1.001	stats.c anomaly check fires	Netlink ANOMALY message emitted to GUI
T+1.010	GUI netlink listener receives message	AlertPopup: "PORT_SCAN: nmap (PID 12847)"
T+1.010	TrayIcon anomaly badge incremented	Badge shows "1" in system tray
T+1.500	Anomaly visible in Anomalies tab	Row: nmap | PORT_SCAN | 500 ports/s | threshold 15
T+2.000	Second 1-second window counted	Anomaly flag remains set, count increments

Table 4.9 — Port scan detection timeline

The analyst can respond immediately by clicking the Block button on the nmap process, which inserts an iptables OUTPUT -m owner --pid-owner 12847 -j DROP rule, halting the scan within 50 ms. Total time from first offending packet to active countermeasure is under 2 seconds.

4.10 Case Study 2: Detecting High Bandwidth Exfiltration

A malicious process (simulated by a Python script named 'systemd-update') opens a TCP connection to an external server and begins uploading data at 15 MB/s — exceeding the HIGH_BW threshold of 10 MB/s.

Time (s)	Event	KTA Response
T+0	Malicious process starts; TCP ESTABLISHED	Connection entry created: PID=9934, state=ESTABLISHED
T+0.5	Upload rate reaches 15 MB/s	rate_out_bps = 15,728,640 (> 10,485,760 threshold)
T+1.0	HIGH_BW anomaly flag set	Netlink ANOMALY: "HIGH_BW: systemd-update (PID 9934)"
T+1.01	AlertPopup shown	"systemd-update uploading 15 MB/s to 185.220.101.x"
T+1.01	RouteMap animates new arc	Bezier arc from Bhopal to destination country visualised

Time (s)	Event	KTA Response
T+3	Analyst inspects connection	Domain: (none — hardcoded IP), Dest country: DE (Germany)
T+4	Analyst clicks 'Block'	iptables DROP rule added; upload stops immediately
T+60	Historical record in HistoryDB	Exfiltration event persisted to SQLite for forensic review

Table 4.10 — High bandwidth exfiltration detection timeline

4.11 Case Study 3: BGP Route Change Detection

BGP route changes are simulated by modifying /etc/hosts to redirect news.example.com from its normal IP (ASN 12345, India) to a different IP (ASN 67890, Russia), simulating the effect of a BGP hijacking event.

Time	Event	KTA BGP Monitor Response
Day 1–7	Normal routing: ASN 12345, IN	Baseline established in BgpMonitor
Day 8, T+0	/etc/hosts modified to redirect domain	Next DNS response returns new IP
Day 8, T+30	New route trace completed for new IP	ASN 67890 (RU) detected — differs from baseline
Day 8, T+31	BGP Monitor comparison fires	Alert: "BGP CHANGE: ASN 12345→67890, IN→RU"
Day 8, T+31	BGPMonitor tab shows alert row	Old ASN: 12345 (IN), New ASN: 67890 (RU), logged

Table 4.11 — BGP route change detection timeline

4.12 Limitations Observed During Testing

- CONN_BURST false positives from package managers: apt and snap were observed opening 25–40 parallel connections during package update checks, briefly exceeding the

CONN_BURST threshold. Addressed by adding apt and snap to the whitelist for CONN_BURST only.

- DNS-over-HTTPS blind spot: Firefox in its default configuration uses DNS-over-HTTPS through Cloudflare (1.1.1.1:443). These DNS queries are indistinguishable from regular HTTPS traffic at the packet level, causing approximately 12% of Firefox connections to have no DNS enrichment.
- traceroute timing variability: On the 4G hotspot, traceroute completion times ranged from 8 to 94 seconds depending on network congestion, delaying new route results by up to 3 minutes during peak traffic.
- Kernel module compilation sensitivity: The module must be compiled against the exact running kernel version. DKMS integration would automate recompilation but was not implemented in the current version.

4.13 Discussion and Analysis

The experimental results validate all seven objectives defined in Chapter 1. Objective 1 (kernel-level packet interception) is confirmed by zero dropped packets across 84,312 tracked connections. Objective 2 (per-process attribution) is confirmed by 96.8% PID attribution accuracy with < 0.05% CPU overhead. Objective 3 (in-kernel DNS parsing) is confirmed by 94.0% immediate domain enrichment. Objective 4 (route daemon) is confirmed by 96% successful route tracing through 4G NAT. Objective 5 (anomaly detection) is confirmed by sub-2-second detection latency across all five anomaly types. Objective 6 (GUI) is confirmed by stable 2.1% CPU usage across all 15 tabs. Objective 7 (integrated remediation) is confirmed by < 50 ms iptables rule application and < 80 ms tc throttle application.

CHAPTER 5 Conclusions and Future Work

5.1 Conclusions

This project designed, implemented, and evaluated the Kernel Traffic Analyzer (KTA) v7.0 — a production-grade Linux network observability system. The following conclusions are drawn from the experimental results presented in Chapter 4.

Conclusion 1 — Kernel-level monitoring is superior to userspace polling: The 96.8% PID attribution accuracy (99.1% for TCP connections > 500 ms) and zero dropped connections over 84,312 tracked flows demonstrate that netfilter-based capture provides fundamentally more complete visibility than /proc polling.

Conclusion 2 — In-kernel DNS parsing enables near-instant enrichment: The 94.0% immediate domain enrichment rate validates the design decision to parse DNS responses in the netfilter hook before connection tracking runs. This approach requires only 312,847 DNS response parsings over 72 hours — well within the performance budget of a softirq hook.

Conclusion 3 — TCP traceroute on port 443 is effective for 4G environments: The 96% successful route tracing rate confirms that traceroute -T -p 443 is the correct choice for environments with carrier-grade NAT.

Conclusion 4 — Threshold-based anomaly detection achieves practical accuracy: False positive rates below 1.5% for all five anomaly types, combined with sub-2-second detection latency, meet the design objectives.

Conclusion 5 — The three-layer architecture achieves excellent resource efficiency: The kernel module overhead of less than 0.05% CPU at 100 Mbps and the GUI's 48 MB stable RSS with 2.1% average CPU demonstrate that a comprehensive monitoring system need not impose significant performance penalties.

Conclusion 6 — Integrated remediation significantly reduces response time: The case studies demonstrate that the combination of real-time anomaly alerts, route visualisation, and one-click iptables/tc control enables a response time from detection to countermeasure of under 5 seconds.

5.2 Future Work

The following eight directions are identified for future enhancement based on limitations observed during testing:

- Full IPv6 support: Extend flow_key and traffic_entry to store 128-bit IPv6 addresses using two uint64_t fields. Estimated effort: 3–5 days.
- Machine learning anomaly detection: Replace fixed-threshold detectors with an online Isolation Forest model trained on per-process behavioural baselines. This would reduce CONN_BURST false positives from browser traffic.
- Generic Netlink family: Replace NETLINK_USERSOCK with a properly registered Generic Netlink family using genl_register_family() for better multi-process subscription management.
- Container-aware monitoring: Add network namespace awareness using struct net* from skb->dev->nd_net to distinguish Docker/Podman containers.
- eBPF co-implementation: Provide an eBPF alternative implementation for environments where loading unsigned kernel modules is prohibited (Secure Boot, locked-down kernels).
- Threat intelligence integration: Replace the heuristic ThreatIntel scorer with live feeds from AbuseIPDB or Shodan's InternetDB for real-time reputation-based scoring.
- Distributed monitoring: Add a lightweight protobuf-based aggregation protocol allowing multiple KTA instances to report to a central server.
- Web dashboard: Export the live data stream via WebSocket from a small embedded HTTP server to enable browser-based monitoring without requiring Qt6 installation.

5.3 Lessons Learned

The development of KTA over seven major iterations yielded several valuable engineering lessons. The most significant was the importance of understanding kernel context discipline. The bug class of spin_lock() called from softirq context (which should always be spin_lock_bh()) caused several hours of debugging in early versions when the system would intermittently deadlock under high packet rates.

The second major lesson was the removal of get_mm_exe_file() in kernel 6.7. This API had been available since the earliest days of the kernel and was used in dozens of tutorials and example kernel modules. Its removal without deprecation warning broke any module that used it. The replacement pattern (rcu_dereference(mm->exe_file) under RCU read lock) is cleaner and correct, but discovering it required reading the kernel commit history.

The third lesson was the importance of the SparklineDelegate vtable constraint. Adding Q_OBJECT to a QStyledItemDelegate subclass used as a delegate for a QTableView column caused a cryptic vtable linker error at compile time. The fix (removing Q_OBJECT and implementing the delegate without signals/slots) is described in Qt developer forums but not in the official documentation. This cost approximately 4 hours of debugging.

5.4 Project Timeline

Phase / Version	Duration	Key Deliverables
v1.0 — Proof of Concept	Weeks 1–3	Basic netfilter hook, /proc/traffic_analyzer with 8 columns, no PID attribution
v2.0 — PID Attribution	Weeks 4–7	sock_cache, inode_cache, flow_cache, resolver workqueue. Attribution accuracy ~72%
v3.0 — DNS Integration	Weeks 8–11	dns_parser.c RFC 1035 implementation, dns_map, DNS enrichment. 22-column proc format
v4.0 — Route Daemon	Weeks 12–15	ta_route_daemon.py, MaxMind integration, route_store, /proc/traffic_analyzer_routes
v5.0 — Qt6 GUI	Weeks 16–22	Full Qt6 GUI with 15 tabs, TrafficModel, ProcModel, RouteMapWidget, HistoryDB
v6.0 — Anomaly Detection	Weeks 23–27	5 anomaly types, velocity_tracker, AlertPopup, TrayIcon, FirewallManager, BandwidthThrottler
v7.0 — Netlink + Polish	Weeks 28–34	Netlink multicast, DnsLeakTab, BgpMonitor, PDF/JSON/CSV export, dark theme, documentation
Testing & Documentation	Weeks 35–56	72-hour test run, case studies, thesis writing, GitHub cleanup

Table 5.1 — KTA development timeline

5.5 Comparison with Project Objectives

Objective	Status	Evidence
1. Kernel-level TCP/UDP interception without packet drops	**Met** √	84,312 connections tracked; 0 dropped packets
2. Per-process attribution via 4-level cache	**Met** √	96.8% accuracy; 71.2% sock_cache hit rate
3. In-kernel DNS parsing (RFC 1035)	**Met** √	94.0% immediate enrichment; 312,847 responses parsed

Objective	Status	Evidence
4. Route daemon with MaxMind geo-enrichment	**Met** √	96% of IPs traced; 98% ASN resolution
5. Five anomaly detectors with < 2 s latency	**Met** √	All 5 types detected; false positives < 1.5%
6. Qt6 GUI with 15 tabs, 1 s refresh	**Met** √	2.1% CPU; 48 MB RSS; all 15 tabs functional
7. Integrated iptables/tc remediation	**Met** √	< 50 ms block; < 80 ms throttle application

Table 5.2 — Objectives achievement summary

5.6 Final Remarks

KTA v7.0 represents the successful completion of a complex, multi-layer systems project that spans kernel programming, network engineering, geographic data processing, and modern GUI development. All seven project objectives have been met. The system has been validated through a 72-hour continuous test with quantitative measurements across all major subsystems.

The project has been open-sourced on GitHub (https://github.com/Anilkumar64/kernal-traffic-analyzer) to enable other developers and security researchers to use, study, and extend the codebase.

The author acknowledges the guidance of Prof. [Supervisor Name] at [Your University Name], whose direction during the project definition phase was instrumental in scoping the system to a feasible yet academically significant set of objectives. The Linux kernel developer community's extensive documentation, kernel.org source code, and the netfilter mailing list archives were invaluable references throughout the implementation.

CHAPTER 6 Implementation Details

6.1 Kernel Module Packet Processing Pipeline

The packet_parser.c file implements a 10-step pipeline that processes every TCP and UDP packet seen by the netfilter hooks. Each step is designed to fail gracefully, returning NF_ACCEPT to ensure that any processing error does not affect packet delivery.

#	Function / Operation	Key Detail
1	IP version check	Read skb->data[0] >> 4 for IP version. Use this rather than skb->protocol, which may not be set correctly for all driver types.
2	Header pointer validation	skb_header_pointer() with sizeof(struct iphdr) to safely read IP header without assuming linear skb layout.
3	Protocol filter	Only process IPPROTO_TCP and IPPROTO_UDP; all others return immediately with NF_ACCEPT.
4	Port extraction	skb_header_pointer() for transport header; extract src/dst port from TCP or UDP header struct.
5	Flow key construction	Populate flow_key with {src_ip, dst_ip, src_port, dst_port, proto}. Call make_canonical() to normalise direction.
6	Existing entry lookup	Locked search of traffic_list for matching flow_key. If found, update stats and return.
7	Socket lookup	skb->sk provides struct sock* for outgoing packets. sk_to_pid() consults sock_cache; on miss, queues workqueue item.
8	PID waterfall	sock_cache → inode_cache → flow_cache → async resolver. Each level queues workqueue item on miss.
9	New entry creation	kzalloc(GFP_ATOMIC) for new traffic_entry. Populate all fields. Atomic insert into traffic_list.
10	Stats and anomaly update	Call stats_update(entry, direction, payload_len). Increment per-process counters. Check velocity_tracker thresholds.

Table 6.1 — packet_parser.c 10-step processing pipeline

A critical implementation detail is the use of skb_header_pointer() rather than direct pointer arithmetic on skb->data. The kernel socket buffer may be non-linear (fragmented across multiple memory pages, common for large packets assembled via GRO). skb_header_pointer() handles non-linear skbs transparently, providing a pointer to a linear copy in a provided buffer when necessary.

6.2 TCP Connection State Machine

The TCP FSM in stats.c implements a simplified version of the RFC 793 state machine with five states relevant to per-connection monitoring: UNKNOWN, SYN_SENT, ESTABLISHED, FIN_WAIT, and CLOSED.

Current State	Observed Packet	New State	Action
UNKNOWN	TCP SYN (outgoing)	SYN_SENT	Increment syn_pending_count
UNKNOWN	TCP SYN (incoming)	SYN_SENT	Record as passive open
SYN_SENT	TCP SYN+ACK	ESTABLISHED	Decrement syn_pending_count; emit CONN_NEW netlink
ESTABLISHED	TCP data packet	ESTABLISHED	Update bytes/packets/rate
ESTABLISHED	TCP FIN	FIN_WAIT	Note fin_seen timestamp
FIN_WAIT	TCP FIN+ACK	CLOSED	Set closed_at; emit CONN_CLOSED netlink
Any	TCP RST	CLOSED	Immediate CLOSED; set closed_at; emit CONN_CLOSED
CLOSED	(any)	CLOSED (eviction pending)	Entry retained for 30 s then evicted

Table 6.2 — TCP connection state machine transitions

6.3 DNS Parser RFC 1035 Implementation

The dns_parser.c implementation follows RFC 1035 Section 4 wire format precisely. The DNS message structure consists of a 12-byte fixed header followed by Question and Answer sections. The parser skips the Question section and processes the Answer section for IN-class A records.

Label encoding in RFC 1035 uses a length-byte followed by label octets, with a zero byte terminating the name. Pointer compression allows a 2-byte value with the top two bits set (0xC0) to encode an offset into the DNS message where the remainder of the name continues. KTA's parser follows up to 10 pointer jumps before aborting with an error, preventing pointer-loop denial-of-service attacks against the kernel.

Field	Offset	Size	Description
ID	0	2 bytes	Transaction ID matching query
FLAGS	2	2 bytes	QR(1) Opcode(4) AA TC RD RA Z(3) RCODE(4)
QDCOUNT	4	2 bytes	Number of questions (typically 1)
ANCOUNT	6	2 bytes	Number of answer RRs
NSCOUNT	8	2 bytes	Number of authority RRs
ARCOUNT	10	2 bytes	Number of additional RRs
QNAME	12	Variable	Question name (length-prefixed labels)
QTYPE	After QNAME	2 bytes	Query type (A=1, AAAA=28, etc.)
QCLASS	After QTYPE	2 bytes	Query class (IN=1)
Answer RRs	After question	Variable	NAME|TYPE|CLASS|TTL|RDLENGTH|RDATA per RR

Table 6.3 — RFC 1035 DNS message header structure

6.4 Route Store Protocol

The route_store subsystem implements a custom protocol for multi-line writes from the Python daemon to /proc/traffic_analyzer_routes. The protocol is: (1) write DEST {ip}\n to set the current destination; (2) write HOP {n}|{ip}|...fields...\n for each hop; (3) write STATUS {DONE|FAILED}\n to finalise the entry. The single ta_write() handler in proc_interface.c dispatches each line to the appropriate handler based on the first token.

6.5 GUI ProcReader Column Mapping

ProcReader.cpp reads all /proc files and parses pipe-delimited lines into C++ data structures. A key subtlety is the make_canonical() normalisation in the kernel module: because the kernel stores the lower IP in the src field regardless of direction, ProcReader must apply the inverse transformation when displaying connections. This logic is implemented in ProcReader::normaliseDirection() which is called for each row during parsing.

SparklineDelegate renders a miniature line chart for the RATE_OUT_BPS and RATE_IN_BPS columns. It maintains a 30-sample ring buffer per connection entry. The delegate must NOT have Q_OBJECT because it inherits from QStyledItemDelegate which already has a virtual table; adding Q_OBJECT creates a vtable conflict that manifests as a linker error.

6.6 Netlink Event Emission

Netlink messages are emitted from netlink_comm.c using nlmsg_new() and genlmsg_put() with GFP_ATOMIC (since emission can occur from softirq context during stats_update). The throttle logic is implemented as a per-entry timer: a CONN_UPDATE message is sent only if the rate has changed by more than 10% since the last message, or 5 seconds have elapsed since the last forced update. This reduces netlink traffic by approximately 95% compared to emitting on every packet.

CHAPTER 7 System Testing and Validation

7.1 Unit Testing

Kernel module unit testing uses two approaches. The first is a userspace shim: key algorithmic functions are compiled into a standalone C program that exercises them with synthetic inputs and checks outputs. This approach was used for dns_parser.c, make_canonical(), and the anomaly threshold logic. The second approach is in-kernel assertion macros (WARN_ON()) at key invariant points.

Test Module	Test Cases	Method	Pass Rate
dns_parser.c	48 test inputs (valid/malformed /compressed)	Userspace shim	100%
make_canonical()	100 flow pairs (random + boundary)	Userspace shim	100%
Anomaly thresholds	25 boundary conditions per anomaly type	Userspace shim	100%
route_store protocol	20 write sequences (valid/partial/con current)	Userspace shim	100%
Cache eviction logic	10 table-full scenarios	In-kernel WARN_ON	100%

Table 7.1 — Unit test results summary

7.2 Integration Testing

Test Scenario	Expected Outcome	Result
Load module, open GUI, browse to google.com	GUI shows Chrome process with DNS domain populated	**Pass** √

Test Scenario	Expected Outcome	Result
Kill and restart route daemon while GUI is open	Routes resume after daemon restart; no GUI crash	**Pass** √
Open 2,000 parallel connections (nc)	Connection table fills to 2,048; oldest CLOSED entries evicted	**Pass** √
Run nmap port scan; verify AlertPopup	AlertPopup appears within 2 s; TrayIcon badge = 1	**Pass** √
Apply iptables block via GUI; verify connectivity loss	Blocked process connections time out; rule visible in iptables -L	**Pass** √
Apply tc throttle via GUI; measure effective rate	iperf3 shows rate ≤ configured limit	**Pass** √
Export to PDF; verify content	PDF contains correct connection data at time of export	**Pass** √

Table 7.2 — Integration test results

7.3 Stress Testing

Test	Load	Duration	Observation
High packet rate (iperf3 loopback)	100 Mbps synthetic traffic	10 min	CPU overhead < 0.05%; no dropped packets; stats_update latency < 2 µs
High connection rate	1,000 new TCP conns/sec (custom script)	5 min	Connection table reached peak 1,980 entries; eviction triggered correctly
Concurrent DNS flood	10,000 DNS responses/sec (crafted payloads)	1 min	dns_map fill rate matched insert rate; no kernel panic; 3 malformed packets handled
Route daemon restart loop	Kill/restart every 5 seconds	30 min	No route_store corruption; GUI displayed partial routes during restart periods
Memory pressure test	Allocate 90% of	10 min	GFP_ATOMIC allocations began failing

Test	Load	Duration	Observation
	RAM via stress-ng		at 92% RAM usage; failures handled gracefully

Table 7.3 — Stress test results

7.4 Edge Case Analysis

The following edge cases were specifically designed and tested to verify correctness of the most sensitive code paths:

- DNS pointer loop: A crafted DNS response with circular pointer compression was fed to the kernel hook. The 10-jump guard correctly terminated parsing and logged a warning to dmesg without crashing.
- RST packet on CLOSED connection: A TCP RST packet arriving for an already-CLOSED connection entry triggered the CLOSED → CLOSED transition with no effect on the entry. No double-free or invalid state.
- PID 0 (kernel thread) attribution: Packets generated by kernel threads have no userspace PID. KTA correctly stores PID=0 and labels these as "[kernel]" in the GUI.
- Connection table full + new connection: When the 2,048 entry limit was reached, new connections caused the oldest CLOSED entry to be evicted. Eviction correctly released memory and removed the entry from all caches.
- Concurrent route writes: Two simultaneous Python daemon instances both attempted to write to /proc/traffic_analyzer_routes. The write mutex in proc_interface.c serialised the writes correctly; no data corruption.

References

[1] V. Jacobson, C. Leres, and S. McCanne, "tcpdump," LBNL Network Research Group, 1989. [Online]. Available: https://www.tcpdump.org

[2] G. Combs, "Wireshark Network Protocol Analyzer," 1998. [Online]. Available: https://www.wireshark.org

[3] R. Russell, "Linux Netfilter Hacking HOWTO," Netfilter Core Team, 1999. [Online]. Available: https://www.netfilter.org

[4] V. Jacobson, "Traceroute," Lawrence Berkeley National Laboratory, 1987. [Online]. Available: https://traceroute.sourceforge.net

[5] S. McCanne and V. Jacobson, "The BSD Packet Filter: A New Architecture for User-level Packet Capture," in Proc. USENIX Winter Technical Conference, San Diego, CA, 1993, pp. 259–269.

[6] B. Gregg, BPF Performance Tools: Linux System and Application Observability. Addison-Wesley Professional, 2019, ISBN 978-0-13-655482-1.

[7] D. E. Denning, "An Intrusion-Detection Model," IEEE Transactions on Software Engineering, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[8] S. Staniford, J. A. Hoagland, and J. M. McAlerney, "Practical Automated Detection of Stealthy Portscans," Journal of Computer Security, vol. 10, no. 1–2, pp. 105–136, 2002.

[9] D. J. Bernstein, "SYN Cookies," 1996. [Online]. Available: http://cr.yp.to/syncookies.html

[10] Cisco Systems, "NetFlow Services Solutions Guide," Cisco IOS Release 12.3, 2004.

[11] F. Weimer, "Passive DNS Replication," in Proc. FIRST Conference on Computer Security Incident Handling, Singapore, 2005.

[12] M. Lad et al., "PHAS: A Prefix Hijack Alert System," in Proc. USENIX Security Symposium, Vancouver, BC, Canada, 2006.

[13] IETF, RFC 7011 — Specification of the IP Flow Information Export (IPFIX) Protocol, 2013.

[14] IETF, RFC 6480 — An Infrastructure to Support Secure Internet Routing (RPKI), 2012.

[15] P. Mockapetris, RFC 1035 — Domain Names — Implementation and Specification, IETF, 1987.

[16] MaxMind Inc., "GeoLite2 Free Geolocation Data," 2002–2025. [Online]. Available: https://dev.maxmind.com/geoip/geolite2-free-geolocation-data

[17] The Qt Company, "Qt 6 Documentation," 2021–2025. [Online]. Available: https://doc.qt.io/qt-6/

[18] Linux Kernel Community, "Netfilter Documentation," kernel.org, 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/networking/netfilter.html

[19] J. Salmela, "Pi-hole: A Black Hole for Internet Advertisements," GitHub, 2014. [Online]. Available: https://github.com/pi-hole/pi-hole

[20] A. K. Reddy, "Kernel Traffic Analyzer v7.0 — Source Code Repository," GitHub, 2025. [Online]. Available: https://github.com/Anilkumar64/kernal-traffic-analyzer

[21] F. Y. Liu, K. M. Ting, and Z. H. Zhou, "Isolation Forest," in Proc. IEEE 8th Int. Conf. Data Mining (ICDM), Pisa, Italy, 2008, pp. 413–422.

[22] Y. Mirsky et al., "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," in Proc. NDSS Symposium, San Diego, CA, 2018.

[23] A. V. Engelen, "nethogs: Per-process Bandwidth Monitoring," GitHub, 2004. [Online]. Available: https://github.com/raboof/nethogs

[24] P. Warren, "iftop: Display Bandwidth Usage on an Interface," 2002. [Online]. Available: http://www.ex-parrot.com/~pdw/iftop/

[25] D. Taht and J. Morton, "RFC 8290 — The Flow Queue CoDel Packet Scheduler and AQM Algorithm," IETF, 2018.

[26] Linux Kernel Community, "Linux Kernel Documentation — workqueue," kernel.org, 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/core-api/workqueue.html

[27] M. McKusick and G. Neville-Neil, The Design and Implementation of the FreeBSD Operating System, 2nd ed. Pearson Education, 2004.

[28] R. Love, Linux Kernel Development, 3rd ed. Addison-Wesley Professional, 2010, ISBN 978-0-672-32946-3.

[29] Verizon, "2023 Data Breach Investigations Report," Verizon Business, 2023. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[30] IBM Security, "Cost of a Data Breach Report 2023," IBM Corporation, 2023. [Online]. Available: https://www.ibm.com/reports/data-breach

[31] T. Bird, "Netfilter's Connection Tracking System," netfilter.org technical paper, 2004.

[32] IETF, RFC 4033 — DNS Security Introduction and Requirements (DNSSEC), 2005.

APPENDIX A Installation Guide

A.1 Prerequisites

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential linux-headers-$(uname -r) cmake \
  qt6-base-dev libqt6printsupport6-dev libsqlite3-dev python3 python3-pip
traceroute
pip3 install geoip2 --break-system-packages
# Download MaxMind GeoLite2 databases (requires free account)
# Place GeoLite2-City.mmdb and GeoLite2-ASN.mmdb in /usr/share/GeoIP/
```

A.2 Build and Load Kernel Module

```
git clone https://github.com/Anilkumar64/kernal-traffic-analyzer.git
cd kernal-traffic-analyzer/kernel_module
make -C /lib/modules/$(uname -r)/build M=$(pwd) modules
sudo insmod src/traffic_analyzer.ko
# Verify module loaded:
lsmod | grep traffic_analyzer
dmesg | tail -20
```

A.3 Start Route Daemon

```
cd ../route_daemon
sudo python3 ta_route_daemon.py &
# Verify daemon is polling:
tail -f /var/log/syslog | grep ta_route
```

A.4 Build and Launch Qt6 GUI

```
cd ../gui
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
sudo ./KernelTrafficAnalyzer
```

A.5 Uninstall

```
sudo rmmod traffic_analyzer
kill $(pgrep -f ta_route_daemon)
```

APPENDIX B /proc File Column Reference

B.1 /proc/traffic_analyzer (22 columns)

Col #	Name	Type	Description
1	PID	int	Process ID (0 = unresolved)
2	UID	int	User ID of socket owner
3	PROCESS	str	Process comm name (max 15 chars)
4	RESOLVED	0/1	1 = PID attribution confirmed
5	STATE	str	TCP state or UDP
6	DNS	0/1	1 = domain from dns_map
7	PROTO	str	TCP or UDP
8	SRC_IP	IPv4	Source IP (canonical: lower IP)
9	DEST_IP	IPv4	Destination IP
10	SRC_PORT	int	Source port
11	DEST_PORT	int	Destination port
12	DOMAIN	str	Domain name or empty
13	OUT_BYTES	long	Total outgoing bytes
14	IN_BYTES	long	Total incoming bytes
15	OUT_PKTS	long	Total outgoing packets
16	IN_PKTS	long	Total incoming packets
17	RATE_OUT_BPS	long	1-second outgoing rate (bytes/s)

Col #	Name	Type	Description
18	RATE_IN_BPS	long	1-second incoming rate (bytes/s)
19	FIRST_SEEN	unix_ts	Connection creation timestamp
20	LAST_SEEN	unix_ts	Last packet timestamp
21	DURATION	secs	LAST_SEEN − FIRST_SEEN
22	CLOSED_AT	unix_ts	Close timestamp (0 if open)

Table B.1 — /proc/traffic_analyzer columns

B.2 /proc/traffic_analyzer_routes (15 columns per hop)

Col #	Name	Description
1	DEST_IP	Final destination IP of the route
2	DOMAIN	Domain name for destination (from dns_map)
3	STATUS	PENDING / RUNNING / DONE / FAILED
4	TOTAL_HOPS	Total number of hops in this route
5	HOP_N	This hop's sequence number (1-based)
6	HOP_IP	IP address of this hop's router
7	HOST	Reverse DNS of hop IP (if available)
8	RTT_MS	Round-trip time to this hop (milliseconds)
9	CITY	MaxMind city name for hop IP
10	COUNTRY	MaxMind country name for hop IP
11	CC	ISO 3166-1 alpha-2 country code

Col #	Name	Description
12	LAT_E6	Latitude × 10^6 (integer, avoids float in kernel)
13	LON_E6	Longitude × 10^6 (integer)
14	ASN	Autonomous System Number for hop IP
15	ORG	Organisation name from MaxMind ASN database

Table B.2 — /proc/traffic_analyzer_routes columns

APPENDIX C Netlink Message Structure Reference

This appendix documents the binary struct layout for KTA's netlink multicast messages. All structs are defined in kernel_module/src/netlink_comm.h. Integers are little-endian (native kernel byte order).

```
/* Netlink configuration */
#define KTA_NETLINK_PROTO 2  /* NETLINK_USERSOCK, proto 2 */
#define KTA_GROUP_CONNS    1
#define KTA_GROUP_ANOMALIES 2
#define KTA_GROUP_DNS      3
#define KTA_GROUP_ROUTES   4

/* Message types */
#define KTA_MSG_CONN_NEW     1
#define KTA_MSG_CONN_UPDATE  2
#define KTA_MSG_CONN_CLOSED  3
#define KTA_MSG_ANOMALY      4
#define KTA_MSG_DNS_RESOLVED 5
#define KTA_MSG_ROUTE_READY  6

struct ta_msg_conn {
    __u8  msg_type;      /* KTA_MSG_CONN_NEW/UPDATE/CLOSED */
    __u32 pid;
    __u32 uid;
    char  process[16];
    __be32 src_ip;
    __be32 dst_ip;
    __u16 src_port;
    __u16 dst_port;
    __u8  proto;         /* IPPROTO_TCP or IPPROTO_UDP */
    __u8  state;
    char  domain[128];
    __u64 out_bytes;
    __u64 in_bytes;
    __u64 rate_out_bps;
    __u64 rate_in_bps;
    __u64 first_seen;
    __u64 last_seen;
    __u64 closed_at;
};

struct ta_msg_anomaly {
    __u8  msg_type;      /* KTA_MSG_ANOMALY */
    __u32 pid;
    char  process[16];
    __u8  anomaly_type;  /* 1=PORT_SCAN 2=SYN_FLOOD 3=CONN_BURST
4=HIGH_CONNS 5=HIGH_BW */
    __u64 value;
    __u64 threshold;
```

```
    __u64 timestamp;
};
```

Struct	Size (approx.)	Usage
ta_msg_conn	~240 bytes	Sent on every connection state change (new/update/closed)
ta_msg_anomaly	~48 bytes	Sent when anomaly flag is first set for a process
ta_msg_dns	~160 bytes	Sent when A record inserted into dns_map
ta_msg_route	~2,340 bytes (32 hops)	Sent when traceroute completes for a destination

Table C.1 — Netlink message struct sizes and usage